Documentazione della Suite di Test — Albero di Regressione

1. Obiettivi e Ambito del Testing

Il presente documento descrive l'architettura, la matrice di tracciabilità e le schede tecniche dettagliate della suite di test automatizzata realizzata per il progetto **Albero di Regressione** (Java), in conformità al Requisito **R3** (Testing completo con JUnit e Mockito) e alle specifiche di progetto.

La suite persegue i seguenti obiettivi di qualità del software: - **Correttezza funzionale**: validazione di tutte le componenti algoritmiche (induzione dell'albero, calcolo dell'errore quadratico SSE, partizionamento discreto e continuo, predizione interattiva). - **Integrità dell'incapsulamento e polimorfismo**: verifica delle gerarchie ad oggetti (`Attribute` , `Node`) e della serializzazione binaria degli oggetti (`Serializable`). - **Resilienza e gestione delle eccezioni**: verifica di tutti i rami di errore previsti dal dominio applicativo (`TrainingDataException` , `UnknownValueException` , `DatabaseConnectionException` , `EmptySetException`). - **Integrazione reale vs. isolamento**: testing end-to-end reale su database MySQL `MapDB` (tabella `provac`) e su socket TCP (porta effimera), combinato con mock JDBC (`Mockito`) per simulare anomalie e schemi non validi senza richiedere permessi DDL speciali. - **Isolamento dell'input interattivo**: testing controllato dell'acquisizione dati da console (`Keyboard`) tramite iniezione di stream in memoria e riallineamento dello stato statico via reflection.

---

2. Tecnologie e Configurazione della Suite

La suite è integrata nel ciclo di vita Maven attraverso il plugin `maven-surefire-plugin` (versione 3.5.2) e fa uso dei seguenti framework: - **JUnit Jupiter 5.11.4** (`junit-jupiter` , `junit-bom`): supporto all'esecuzione dei test, asserzioni ricche, test annidati (`@Nested`), gestione del ciclo di vita (`@BeforeAll` , `@BeforeEach` , `@AfterEach` , `@AfterAll`) e directory temporanee sicure (`@TempDir`). - **Mockito 5.15.2** (`mockito-core` , `mockito-junit-jupiter`): creazione di mock per oggetti JDBC (`Connection` , `Statement` , `ResultSet` , `ResultSetMetaData`) e intercettazione dei costruttori tramite `Mockito.mockConstruction(DbAccess.class, ...)` .

---

3. Matrice di Tracciabilità Requisiti $\leftrightarrow$ Test

ID Requisito	Modulo	Package	Classe di Test	Metodo di Test	Funzionalità Verificata	Tipologia
REQ-DATA-01	`Server`	`data`	`AttributeTest`	`testContinuousAttributeInitialization`	Costruzione, getter e `toString()` di `ContinuousAttribute`	Unit Test
REQ-DATA-02	`Server`	`data`	`AttributeTest`	`testContinuousAttributeSerialization`	Serializzazione binaria di `ContinuousAttribute`	Unit Test
REQ-DATA-03	`Server`	`data`	`AttributeTest`	`testDiscreteAttributeDeduplicationAndSorting`	Deduplicazione in `TreeSet` e ordinamento lessicografico dei valori	Unit Test
REQ-DATA-04	`Server`	`data`	`AttributeTest`	`testEmptyAndSingleValueDomains`	Gestione di domini discreti vuoti e a singolo elemento	Unit Test
REQ-DATA-05	`Server`	`data`	`AttributeTest`	`testDiscreteAttributeSerialization`	Serializzazione binaria di `DiscreteAttribute` con dominio	Unit Test
REQ-DATA-06	`Server`	`data`	`AttributeTest`	`testPolymorphism`	Trattamento polimorfico tramite la classe astratta `Attribute`	Unit Test
REQ-DATA-07	`Server`	`data`	`DataTest`	`testLoadProvaCSuccessfully`	Caricamento nominale da MySQL `MapDB.provaC` (15 righe, X, Y, C)	Integration Test
REQ-DATA-08	`Server`	`data`	`DataTest`	`testSortDiscreteAttributeFullRange`	Quicksort in-place su attributo discreto X	Unit Test
REQ-DATA-09	`Server`	`data`	`DataTest`	`testSortContinuousAttributeFullRange`	Quicksort in-place su attributo continuo Y	Unit Test
REQ-DATA-10	`Server`	`data`	`DataTest`	`testSortSubRange`	Quicksort limitato al sotto-range `[begin, end]`	Unit Test
REQ-DATA-11	`Server`	`data`	`DataTest`	`testSortBoundaryRanges`	Quicksort con range degenere (`begin == end` e `begin > end`)	Unit Test
REQ-ERR-01	`Server`	`data`	`DataTest`	`testInvalidTableName`	`TrainingDataException` per nome tabella nullo o vuoto	Error Handling
REQ-ERR-02	`Server`	`data`	`DataTest`	`testNonExistentTable`	`TrainingDataException` per tabella inesistente su database	Error Handling

ID Requisito	Modulo	Package	Classe di Test	Metodo di Test	Funzionalità Verificata	Tipologia
REQ-ERR-03	`Server`	`data`	`DataTest`	`testTableWithLessThanTwoColumns`	`TrainingDataException` per tabella con meno di 2 colonne	Mock Test
REQ-ERR-04	`Server`	`data`	`DataTest`	`testLastColumnNotNumeric`	`TrainingDataException` per target di classe non numerico	Mock Test
REQ-ERR-05	`Server`	`data`	`DataTest`	`testNullValuesInTuple`	`TrainingDataException` per presenza di valori `NULL`	Mock Test
REQ-ERR-06	`Server`	`data`	`DataTest`	`testNonNumericValueInNumericColumn`	`TrainingDataException` per tipo incompatibile in colonna numerica	Mock Test
REQ-ERR-07	`Server`	`data`	`DataTest`	`testEmptyTable`	`TrainingDataException` per tabella priva di tuple	Mock Test
REQ-DB-01	`Server`	`database`	`DatabasePackageTest`	`testColumnProperties`	Creazione, denominazione e rilevamento tipo numerico in `Column`	Unit Test
REQ-DB-02	`Server`	`database`	`DatabasePackageTest`	`testExampleAddGetIterator`	Inserimento, accesso posizionale, stringa e iteratore in `Example`	Unit Test
REQ-DB-03	`Server`	`database`	`DatabasePackageTest`	`testExampleCompareTo`	Ordinamento lessicografico `compareTo` tra tuple `Example`	Unit Test
REQ-DB-04	`Server`	`database`	`DatabasePackageTest`	`testInitAndCloseConnection`	Apertura e chiusura connessione JDBC MySQL in `DbAccess`	Integration Test
REQ-DB-05	`Server`	`database`	`DatabasePackageTest`	`testTableSchemaReading`	Introspezione metadati JDBC schema tabella `provac`	Integration Test
REQ-DB-06	`Server`	`database`	`DatabasePackageTest`	`testTableDataGetTransazioni`	Estrazione tuple da tabella con `TableData.getTransazioni`	Integration Test
REQ-DB-07	`Server`	`database`	`DatabasePackageTest`	`testGetDistinctColumnValues`	Query `DISTINCT` ordinata con `TableData.getDistinctColumnValues`	Integration Test
REQ-DB-08	`Server`	`database`	`DatabasePackageTest`	`testTableDataExceptions`	Gestione `SQLException` ed `EmptySetException` in `TableData`	Integration Test
REQ-TREE-01	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testExactSSEZeroVariance`	Calcolo matematico SSE = 0.0 su subset B	Unit Test
REQ-TREE-02	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testExactSSEKnownSubsetA`	Calcolo matematico SSE = 0.625 e media = 1.25 su subset A	Unit Test
REQ-TREE-03	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testLeafNodeProperties`	Proprietà base di `LeafNode` (0 figli, toString con prefisso)	Unit Test
REQ-TREE-04	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testDiscreteNodePartitioning`	Partizionamento multi-ramo `DiscreteNode`, splitVariance e testCondition	Unit Test
REQ-TREE-05	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testContinuousNodePartitioning`	Soglia ottima split binario in `ContinuousNode` e testCondition	Unit Test
REQ-TREE-06	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testContinuousNodeConstantValue`	Gestione caso degenere valore costante in `ContinuousNode`	Unit Test
REQ-TREE-07	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testTreeInductionAndPrints`	Induzione completa `RegressionTree`, `printTree()` e `printRules()`	Unit Test
REQ-TREE-08	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testPredictClassSuccessful`	Predizione interattiva `predictClass` con stream serializzati in memoria	Unit Test
REQ-TREE-09	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testPredictClassUnknownValueException`	Sollevamento di `UnknownValueException` per ramo non valido	Unit Test
REQ-TREE-10	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	`testSerializationRoundTrip`	Salvataggio e caricamento `.dmp` con `salva()` e `carica()` su `@TempDir`	Unit Test
REQ-SOCK-01	`Server`	`server`	`SocketClientServerIntegrationTest`	`testFullFlowAcquireLearnPredict`	Sequenza completa comandi socket 0 -> 1 -> 3 su porta effimera	Integration Test

ID Requisito	Modulo	Package	Classe di Test	Metodo di Test	Funzionalità Verificata	Tipologia
REQ-SOCK-02	`Server`	`server`	`SocketClientServerIntegrationTest`	`testLoadArchiveAndPredictFlow`	Sequenza comandi socket 2 -> 3 (caricamento archivio e predizione)	Integration Test
REQ-SOCK-03	`Server`	`server`	`SocketClientServerIntegrationTest`	`testErrorHandlingOverSocket`	Gestione errori socket (tabella errata, file mancante, comando invalido)	Integration Test
REQ-UTIL-01	`Client`	`utility`	`KeyboardTest`	`testReadStringAndWord`	Acquisizione di righe intere e singole parole da input bufferizzato	Unit Test
REQ-UTIL-02	`Client`	`utility`	`KeyboardTest`	`testReadBoolean`	Parsing booleano "true"/"false" e gestione errori di formato	Unit Test
REQ-UTIL-03	`Client`	`utility`	`KeyboardTest`	`testReadChar`	Acquisizione carattere per carattere	Unit Test
REQ-UTIL-04	`Client`	`utility`	`KeyboardTest`	`testReadInt`	Parsing intero con fallback `Integer.MIN_VALUE` e conteggio errori	Unit Test
REQ-UTIL-05	`Client`	`utility`	`KeyboardTest`	`testReadLong`	Parsing long a 64-bit con fallback `Long.MIN_VALUE`	Unit Test
REQ-UTIL-06	`Client`	`utility`	`KeyboardTest`	`testReadFloatAndDouble`	Parsing float e double con fallback `NaN` e tracciamento errori	Unit Test
REQ-UTIL-07	`Client`	`utility`	`KeyboardTest`	`testEndOfLineAndErrorControl`	Riconoscimento fine riga e controllo visualizzazione/reset errori	Unit Test

4. Schede Tecniche di Dettaglio per Classe di Test

4.1 `data.AttributeTest`

Questa classe valida la gerarchia di classi del package `data` : - **`testContinuousAttributeInitialization`** : verifica che il costruttore assegni correttamente nome e indice numerico, e che `toString()` restituisca il nome dell'attributo. - **`testContinuousAttributeSerialization`** : serializza su `ByteArrayOutputStream` un'istanza di `ContinuousAttribute` e la deserializza con `ObjectInputStream` , asserendo l'uguaglianza dei campi di stato. - **`testDiscreteAttributeDeduplicationAndSorting`** : fornisce valori ripetuti e disordinati (`{"Sunny", "Rain", "Sunny", "Overcast", "Rain"}`) e verifica che l'iteratore restituisca l'insieme deduplicato e ordinato lessicograficamente (`["Overcast", "Rain", "Sunny"]`), verificando inoltre che `getNumberOfDistinctValues()` restituisca `3` . - **`testEmptyAndSingleValueDomains`** : valida i casi limite di domini vuoti e a singolo elemento. - **`testDiscreteAttributeSerialization`** : valida il round-trip di serializzazione e deserializzazione preservando l'integrità del `TreeSet` interno e dell'iteratore `Iterable<String>` . - **`testPolymorphism`** : verifica l'upcasting a tipo astratto `Attribute` e il corretto dispatch dinamico.

4.2 `data.DataTest`

Questa classe testa l'acquisizione, la validazione dello schema e l'ordinamento Quicksort in-place dei dataset: - **`testLoadProvaCSuccessfully`** : effettua una connessione reale a MySQL `MapDB.provac` , verificando l'istanziazione di 15 tuple, la presenza di 2 attributi indipendenti (uno discreto `X` e uno continuo `Y`) e 1 attributo target continuo `C` . - **`testSortDiscreteAttributeFullRange`** : ordina l'intero dataset sull'attributo discreto `X` e verifica l'ordinamento in-place (tutte le 'A' prima delle 'B'). - **`testSortContinuousAttributeFullRange`** : ordina l'intero dataset sull'attributo continuo `Y` e verifica che ogni elemento sia minore o uguale al successivo. - **`testSortSubRange`** : ordina un sotto-intervallo arbitrario `[3, 8]` e verifica che le posizioni esterne rimangano inalterate mentre quelle interne risultino ordinate. - **`testSortBoundaryRanges`** : controlla i casi degenere `begin == end` e `begin > end` confermando la totale stabilità senza eccezioni. - **`testInvalidTableName`** : verifica il rigetto immediato di nomi tabella `null` , vuoti o costituiti da soli spazi con `TrainingDataException` . - **`testNonExistentTable`** : verifica la gestione dell'errore JDBC in presenza di tabella assente. - **`testTableWithLessThanTwoColumns`** : usa `Mockito.mockConstruction` su `DbAccess` per simulare un result set avente 1 sola colonna, verificando il messaggio d'errore richiesto. - **`testLastColumnNotNumeric`** : simula una tabella con target `VARCHAR` , accertando il sollevamento dell'eccezione sul tipo target. - **`testNullValuesInTuple`** : simula la presenza di un valore `null` all'interno di una tupla e verifica il blocco del caricamento. - **`testNonNumericValueInNumericColumn`** : simula una stringa all'interno di una colonna dichiarata numerica, verificando il sollevamento di `TrainingDataException` . - **`testEmptyTable`** : simula una tabella valida ma priva di righe (`rs.next() == false`), verificando l'eccezione dedicata.

4.3 `database.DatabasePackageTest`

Questa classe copre il sottosistema di persistenza relazionale: - **`testColumnProperties`** : controlla il costruttore package-private, il nome, la stringa formattata e il predicato `isNumber()` su tipi testuali e numerici. - **`testExampleAddGetIterator`** : valida la struttura dati `Example` , l'inserimento posizionale, la formattazione testuale e l'iteratore. - **`testExampleCompareTo`** : verifica la relazione d'ordine totale definita dal metodo `compareTo` su coppie di tuple identiche e distinte. - **`testInitAndCloseConnection`** : apre e chiude una connessione reale a MySQL, verificando che la connessione risulti aperta dopo `initConnection()` e chiusa dopo `closeConnection()` . - **`testTableSchemaReading`** : interroga i metadati JDBC di `provac` validando nomi colonne e tipi estratti. - **`testTableDataGetTransazioni`** : estrae le 15 tuple reali come lista di oggetti `Example` . - **`testGetDistinctColumnValues`** : esegue la query `DISTINCT` ordinata su `X`

e `Y`, validando la corrispondenza dei valori univoci (`{"A", "B"}`). - **`testTableDataExceptions`** : verifica che query su tabelle non esistenti propaghino `SQLException` .

4.4 `tree.TreeNodesAndAlgorithmTest`

Questa classe valida la correttezza matematica e l'induzione algoritmica: - **`testExactSSEZeroVariance`** : sul sottoinsieme di 5 esempi con $X='B'$, tutti con $C=10.0$, calcola matematicamente e verifica $SSE = 0.0$ e valore predetto 10.0. - **`testExactSSEKnownSubsetA`** : sul sottoinsieme di 10 esempi con $X='A'$ (cinque 1.0 e cinque 1.5), calcola $\bar{y} = 1.25$ ed $SSE = 0.625$, verificando la perfetta corrispondenza con l'implementazione in `LeafNode` . - **`testLeafNodeProperties`** : verifica che una foglia abbia 0 figli e la stringa inizi con `"LEAF : class="` . - **`testDiscreteNodePartitioning`** : crea un nodo di split discreto su `X` , verifica la creazione di 2 rami, la varianza aggregata di split e il comportamento di `testCondition` su valori noti e sconosciuti. - **`testContinuousNodePartitioning`** : crea un nodo di split continuo su `Y` , verificando la selezione del miglior punto di taglio e il routing dei valori. - **`testContinuousNodeConstantValue`** : gestisce il caso limite in cui l'intervallo contiene valori costanti, generando una singola partizione. - **`testTreeInductionAndPrints`** : induce l'intero albero di regressione su `provac` e verifica `printTree()` e `printRules()` . - **`testPredictClassSuccessful`** : simula la consultazione interattiva collegando stream binari in memoria (`ObjectOutputStream` / `ObjectInputStream`), rispondendo alla query e ottenendo il valore predetto atteso (10.0). - **`testPredictClassUnknownValueException`** : invia una risposta non valida al ramo (`"99"`) e asserisce il sollevamento di `UnknownValueException` . - **`testSerializationRoundTrip`** : serializza l'albero su file `.dmp` temporaneo mediante `salva()` e lo ricarica con `carica()` , verificando l'identità predittiva del modello deserializzato e la gestione di file inesistenti.

4.5 `server.SocketClientServerIntegrationTest`

Questa classe esegue test end-to-end su socket TCP: - **`setUpServer`** / **`tearDownServer`** : alloca un `ServerSocket` su porta effimera `0` ed esegue il dispatcher in un `ExecutorService` , garantendo chiusura immediata e cancellazione dei file `.dmp` . - **`testFullFlowAcquireLearnPredict`** : client TCP che invia sequenzialmente i comandi `0` (acquisizione da DB `provac`), `1` (induzione e salvataggio `provac.dmp`), e `3` (predizione interattiva con risposta al bivio e ricezione dell'esito `OK` con valore predetto `10.0`). - **`testLoadArchiveAndPredictFlow`** : verifica la persistenza cross-client: il primo client acquisisce e crea il dump; il secondo client invia il comando `2` caricando il dump e procede alla predizione con esito corretto. - **`testErrorHandlingOverSocket`** : verifica che richieste di tabelle inesistenti restituiscano la stringa dell'eccezione, che caricamenti di file mancanti restituiscano `FileNotFoundException` e che comandi numerici sconosciuti (`999`) rispondano con `"Invalid command: 999"` .

4.6 `utility.KeyboardTest`

Questa classe valida la classe di utilità del client: - **`captureOriginalIn`** / **`restoreOriginalIn`** : preserva e ripristina lo standard input originale `System.in` . - **`provideInput`** : usa la Java Reflection per reimpostare i campi statici interni `in` , `reader` e `current_token` di `Keyboard` , azzerando i contatori di errore e garantendo l'assoluto determinismo di ogni test. - **`testReadStringAndWord`** : valida l'acquisizione di righe complete e token delimitati da spazi. - **`testReadBoolean`** : verifica il parsing insensibile al maiuscolo/minuscolo e il fallback `false` con incremento contatore errori per token errati. - **`testReadChar`** : legge singoli caratteri in sequenza. - **`testReadInt`** : effettua il parsing di interi positivi, negativi, zero e il fallback a `Integer.MIN_VALUE` . - **`testReadLong`** : effettua il parsing a 64 bit e il fallback a `Long.MIN_VALUE` . - **`testReadFloatAndDouble`** : verifica numeri decimali e il fallback a `Float.NaN` e `Double.NaN` . - **`testEndOfLineAndErrorControl`** : verifica la segnalazione di fine riga, il conteggio degli errori e il toggle `setPrintErrors` .

---

5. Modalità di Esecuzione dei Test

Per eseguire l'intera suite di test con Surefire:

```
mvn -f "pom.xml" clean test
```

Per eseguire selettivamente i test del server o del client:

```
# Modulo Server
mvn -f "Server/pom.xml" test

# Modulo Client
mvn -f "Client/pom.xml" test
```

Per eseguire una singola classe di test specifica (ad esempio `TreeNodesAndAlgorithmTest`):

```
mvn -f "Server/pom.xml" test -Dtest=TreeNodesAndAlgorithmTest
```